

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4
по дисциплине
«Алгоритмы и структуры данных»

Выполнил студент гр. ИВТб-1303-06-00	_____ /Гортоломей И.К./
Проверил заведующий кафедрой ЭВМ	_____ /Долженкова М.Л./

Киров
2026

Цель работы:

Отточить навыки решения задач в условиях ограниченного времени.

Вводные

Для выполнения данной лабораторной работы был выбран Codeforces Round 1096 (Div. 3). <https://codeforces.com/contest/2227>

Описание процесса решения

Было выбрано и запущено виртуальное соревнование. Первые 40 минут соревнования я пропустил, так как пришлось отвлечься и написать контрольную по истории. Далее я открыл задачу А - Кошари <https://codeforces.com/contest/2227/problem/A>, в данной задаче требуется дать ответ yes/no, можно ли добраться из точки (0, 0) в (x, y) имея возможность прибавить 2 к одной из координат за шаг или прибавить 1 к одной из координат один раз. Если мы можем прибавить по 2 сколько угодно раз, то значит мы передвигаемся по чётным, а также имея возможность один раз прибавить 1 мы можем сдвинуться на нечётное по одной из координат, следовательно, сумма остатков от деления x и y на 2 должна быть меньше двух. $x\%2 + y\%2 < 2$.

Листинг 1: Решение задачи А - Кошари

```
1  #include <stdio.h>
2
3  int main() {
4      int t; scanf("%d", &t);
5      for (; t > 0; t--) {
6          int x, y; scanf("%d %d", &x, &y);
7          printf(x%2 + y%2 > 1 ? "no\n" : "yes\n");
8      }; return 0;
9  }
```

<https://codeforces.com/contest/2227/submission/374474673>

После быстрого решения первой задачи, я приступил к решению следующей задачи. В задаче B. Party Monster (<https://codeforces.com/contest/2227/problem/B>) требуется вывести yes/но в зависимости от того, можно ли составить из входящих байт, скобок (открывающая «(» и закрывающая «)») правильную структуру (например «()()» или «(())»). Первым делом я решил проверить, что пройдёт ли код который просто проверяет кол-во открывающих и закрывающих скобок, ведь если их разное кол-во, то не получится составить из них правильную структуру. Быстро написал код который считает скобки $a == b$ и это решение подошло.

Листинг 2: Решение задачи B. Party Monster

```
1  #include <stdio.h>
2
3  int main() {
4      int t; scanf("%d", &t);
5      for (; t > 0; t--) {
6          int n; scanf("%d\n", &n);
7          int a = 0;
8          int b = 0;
9          for (; n > 0; n--) {
10             char x = getchar();
11             if (x == '(') a++;
12             if (x == ')') b++;
13         }
14         printf(a != b ? "no\n" : "yes\n");
15     }
16     return 0;
17 }
```

<https://codeforces.com/contest/2227/submission/374475638>

После решения первых двух заданий осталось примерно 1ч 30м до конца, так что я приступил к следующей задаче.

Следующая задача C - Снегопад (<https://codeforces.com/contest/2227/problem/C>) не показалась простой сразу. Минут 10 не мог понять, что требуется задача и почему получаю неверный ответ. Оказалось, что я просто забыл развести числа, которые делятся на 2 и/или 3 и могут образовывать последовательности делящиеся на 6. Сделал вывод по массиву, чтобы сначала сначала выводить числа, которые делятся на 6, потом числа которые делятся на 2 (но не делятся на 6), за ними «нейтральные» числа, которые не делятся ни на 2, ни на 3, а в конце числа которые делятся на 3 (но не делятся на 6). В итоге я получаю массив, в котором кол-во подмассивов, делящихся на 6 минимальное. Обработка занимает $O(4n) \rightarrow O(n)$.

Листинг 3: Решение задачи C - Снегопад

```
1  #include <stdio.h>
2
3  int main() {
4      int t;
5      scanf("%d", &t);
6
7      while (t--) {
8          int n;
9          scanf("%d", &n);
10
11         int a[n];
12         for (int i = 0; i < n; i++) {
13             scanf("%d", &a[i]);
14         }
15
16         for (int i = 0; i < n; i++) {
17             if (a[i] % 6 == 0) {
18                 printf("%d ", a[i]);
19             }
20         }
21
22         for (int i = 0; i < n; i++) {
23             if (a[i] % 6 != 0 && a[i] % 2 == 0) {
```

```

24     printf("%d ", a[i]);
25 }
26 }
27
28 for (int i = 0; i < n; i++) {
29     if (a[i] % 2 != 0 && a[i] % 3 != 0) {
30         printf("%d ", a[i]);
31     }
32 }
33
34 for (int i = 0; i < n; i++) {
35     if (a[i] % 6 != 0 && a[i] % 3 == 0) {
36         printf("%d ", a[i]);
37     }
38 }
39
40 printf("\n");
41 }
42
43 return 0;
44 }

```

<https://codeforces.com/contest/2227/submission/374479599> В итоге, после решения этой задачи оставалось около 45 минут до конца соревнования, потому что я забыл про делители 2 и 3, и по этому потратил много времени.

Я посмотрел задачу D. ПалиндроМEX (<https://codeforces.com/contest/2227/problem/D>), понял что нужно как-то перебрать все палиндромы в массиве, выполнить функцию mex для каждого палиндрома (половины палиндрома) и найти такой палиндром, у которого функция mex возвращает максимальное число (mex возвращает минимальное отсутствующее число). На обдумывание этой задачи ушло 5-10 минут, так что я решил не пытаться писать алгоритм и перешёл к следующей задаче.

В задаче E. Всё съехало (<https://codeforces.com/contest/2227/problem/E>) нам дают одномерный массив (который как бы представляет из себя матрицу) блоков, и нам нужно сказать сколько блоков сдвинется, когда гравитация поменяется с направления вниз на направление вправо. Данная задача решалась бы просто, если бы не одно НО: нам предоставляется возможность убрать один из кубов, чтобы другие кубы могли сдвинуться и нам нужно выбрать такой куб, после удаления которого сдвинется максимальное число кубов. Я начал писать алгоритм, проверил что подсчёт сдвинувшихся правильный, но не правильно выбирал куб который нужно удалять. Я предполагаю, что где-то в массиве могут быть столбцы в которых много кубов, а между ними столбец с 1-2 кубами и при удалении куба в этом столбце сдвинется много других кубов. Когда я это понял - время соревнования закончилось и я не успел завершить задачу.

Выводы

Проанализировав своё участие в соревновании я пришёл к выводу, что мне нужно больше времени для обдумывания поставленных условий, подбора подходящего алгоритма и проверки кода на ошибки для получения качественного результата. Много времени уходит на промежуточные проверки (компиляция тестов, дебагинг и тд.) потому что мне необходимо «получать обратную связь от кода».

Для ускорения решения задач можно отказаться от промежуточных проверок и заучить синтаксис одного из удобных языков, но «ускорить мышление» в моей ситуации наверное не получится, или это придёт с опытом.